

Relatório 19 - Prática: Pipelines de Dados II - Airflow (III)

Igor Carvalho Marchi

1. Introdução

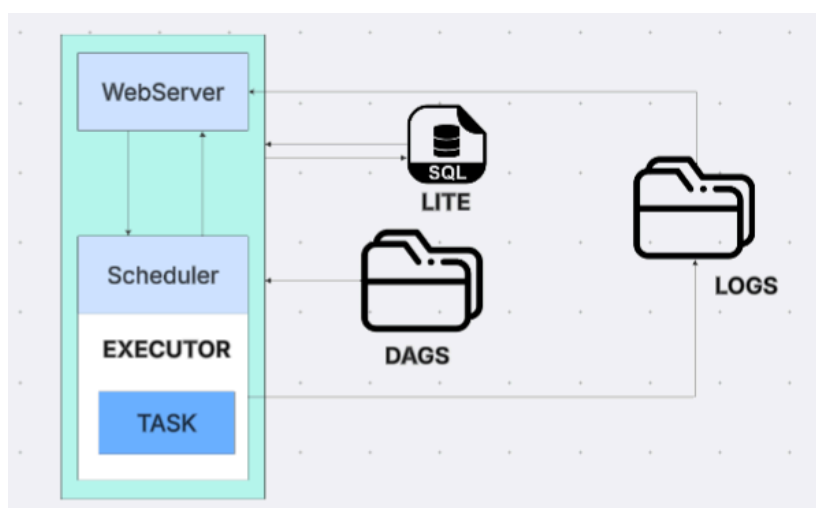
O Card 19 dá continuidade ao aprendizado de Apache Airflow, combinando a revisão de conteúdos essenciais já vistos no Card 12 com a apresentação de recursos inéditos para a gestão de pipelines. Após o acompanhamento das aulas teóricas, o conhecimento foi aplicado na construção de um projeto prático em código, comentado brevemente no fim do relatório.

2. Desenvolvimento

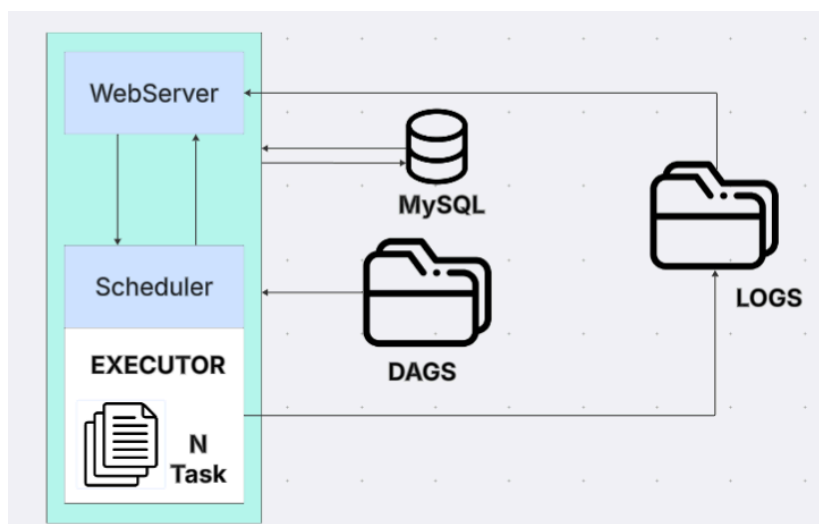
2.1. Executores e Arquitetura de Banco de Dados

A escolha do executor e da base de dados define como o Airflow gerencia e escala suas tarefas

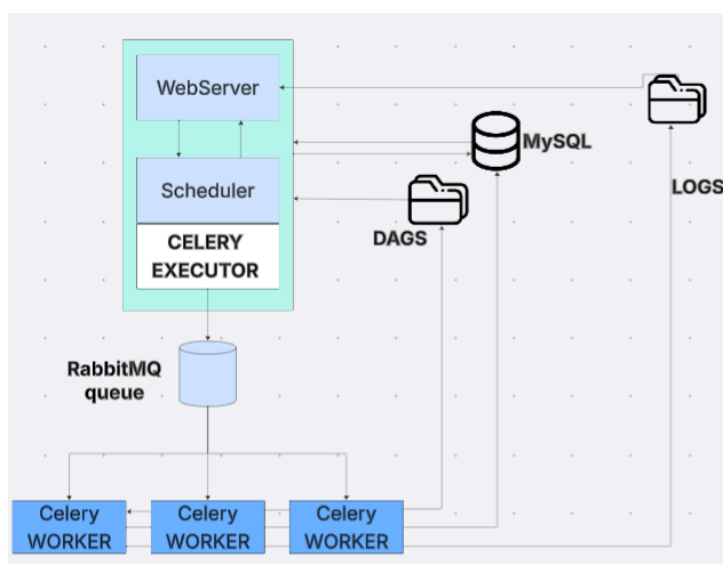
SQLite e Sequential Executor: O SQLite é um banco leve, ideal para cenários de testes e validações. Por ter limitações com operações concorrentes, trabalha em conjunto com o Sequential Executor, que roda apenas uma tarefa por vez sequencialmente.



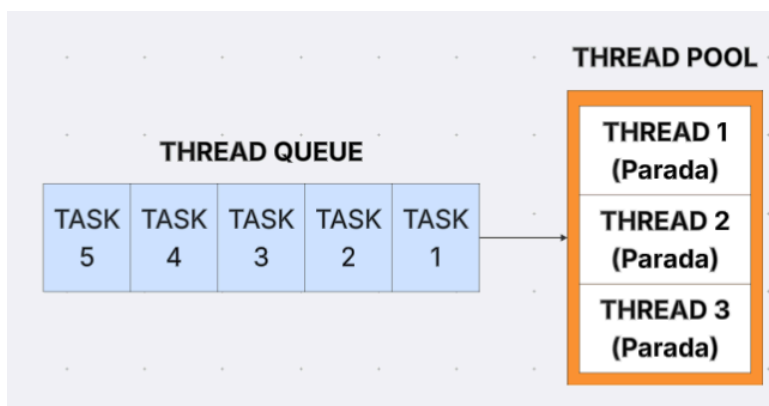
Local Executor: Evolução para cenários locais onde se busca concorrência, permitindo rodar múltiplas tarefas em paralelo na mesma máquina.



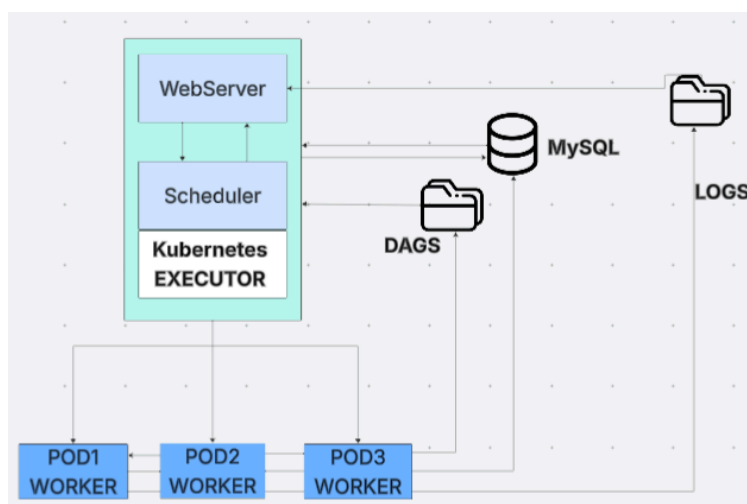
Celery Executor: Permite a distribuição de tarefas entre múltiplos workers. Utiliza um sistema de filas (como RabbitMQ ou Redis) para gerenciar as rotinas enviadas pelos workers.



Pools e Queues: Os Pools limitam a quantidade de tarefas simultâneas para proteger recursos escassos (como APIs com limites de requisições). Já as Queues direcionam tarefas para workers específicos, otimizando máquinas por perfil de processamento.



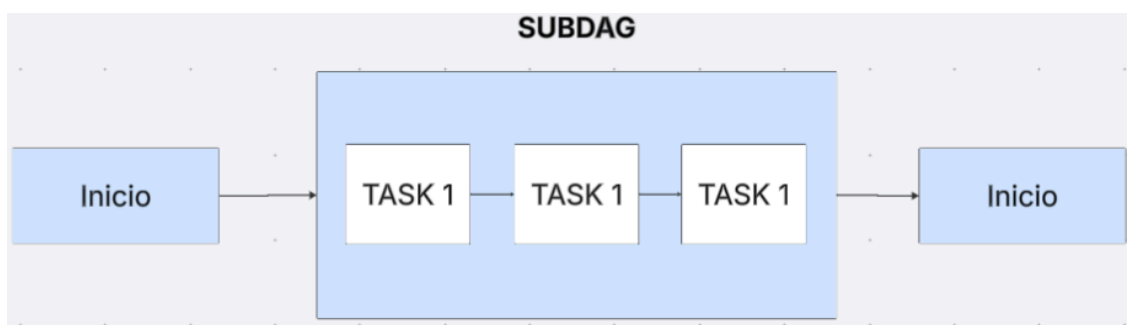
Kubernetes Executor: Cria pods dedicados e dinâmicos para cada tarefa no cluster. Diferente do Celery, cujos workers ficam constantemente alocados, o Kubernetes aloca o recurso sob demanda e o encerra logo após o fim da execução.



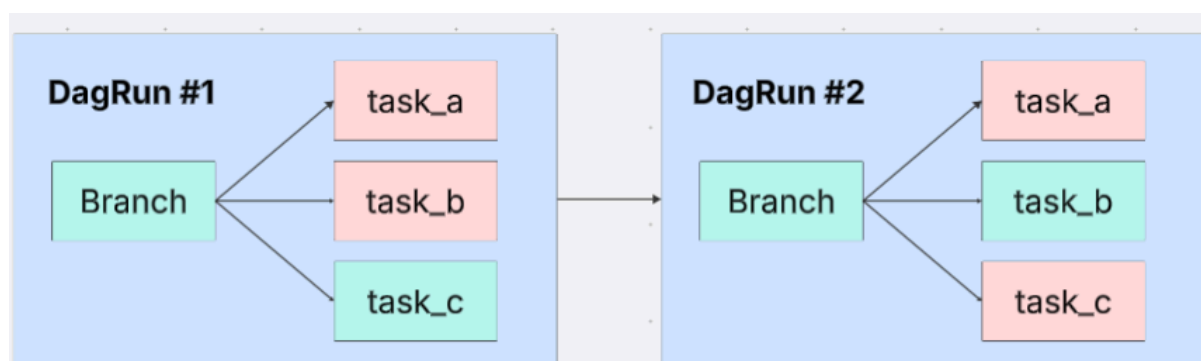
2.2. Fluxos Avançados e Comunicação entre DAGs

Para construir pipelines dinâmicos e interdependentes, explorei os seguintes conceitos:

- **SubDAGs:** Agrupam tarefas relacionadas em um subfluxo, melhorando a organização visual, porém exigindo cuidado com a alocação de slots para evitar quadros de deadlock.



- **Branching:** Utilizado via BranchPythonOperator para definir caminhos condicionais dentro do fluxo, garantindo que o pipeline tome decisões com base na análise de dados em tempo de execução.



- **XCom:** Mecanismo leve para troca de pequenos metadados (como IDs e caminhos de arquivos) entre tarefas isoladas.
- **TriggerDagRunOperator e ExternalTaskSensor:** Utilizados para desacoplar e conectar fluxos. O primeiro inicia a execução de uma segunda DAG de forma remota, enquanto o segundo faz a DAG atual aguardar a conclusão de uma etapa específica em um fluxo externo.

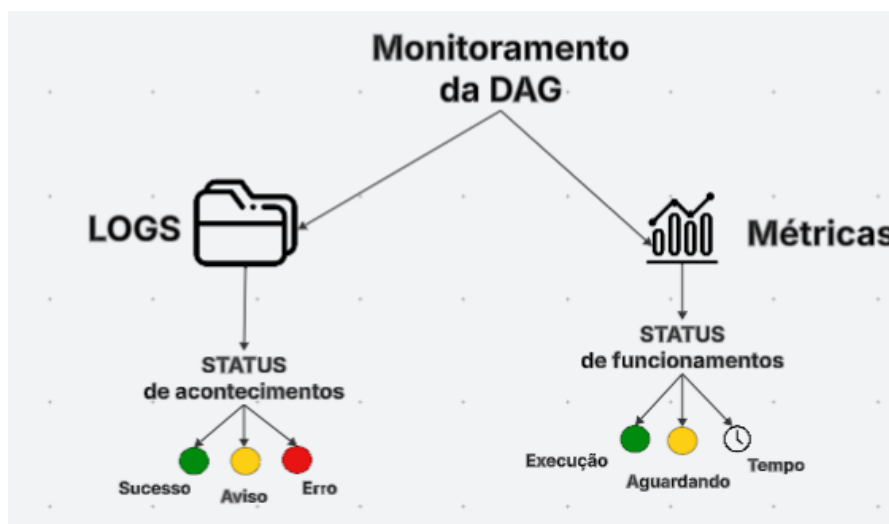
2.7 Branching

Branching permite criar caminhos diferentes dentro de uma DAG, com o BranchPythonOperator, uma função pode analisar determinada condição e retornar qual tarefa deverá ser executada. Assim, nem todas as execuções da DAG precisam seguir exatamente o mesmo caminho. Esse conceito também foi utilizado na minha prática para verificar os sensores da fazenda.

2.4. Monitoramento (Seção 8)

Para garantir a saúde do ambiente, o monitoramento foi dividido em:

- **Logs:** Centralização do registro de execução em serviços como Amazon S3 ou Elasticsearch para evitar que os arquivos fiquem espalhados pelos workers.
- **Métricas:** Coleta de indicadores de sistema via Stack TIG (Telegraf + InfluxDB + Grafana) para acompanhar tempo de execução, fila e tarefas pendentes.



2.5. Segurança (Seção 9)

Por lidar com dados e credenciais sensíveis, apliquei os pilares de segurança do Airflow:

- **Fernet Key:** Criptografia das senhas e conexões salvas no banco. Vi a importância da rotação de chaves sem perder o acesso aos dados existentes.
- **Ocultação de Variáveis e RBAC:** O Airflow oculta valores de tela por padrão caso a chave contenha palavras reservadas. O RBAC (Controle de Acesso Baseado em Funções) organiza permissões por papéis (Admin, User, Viewer).

PRÁTICA

Na atividade prática foi desenvolvido um sistema de monitoramento de uma fazenda utilizando Apache Airflow, dividido em três DAGs responsáveis pelo monitoramento, atendimento de emergências e registro das ocorrências. O sistema simula dados de temperatura, água e ração, analisa os valores e, caso encontre alguma situação fora do normal, aciona automaticamente o fluxo de emergência, realiza as verificações necessárias e registra a ocorrência. A prática utilizou recursos como PythonOperator, BranchPythonOperator, XCom, TaskGroup, EmptyOperator e TriggerDagRunOperator.

3. Conclusão

O Card 19 aprofundou os conhecimentos sobre Apache Airflow, apresentando recursos para execução, distribuição e controle de tarefas. Na prática, foi desenvolvido um sistema de monitoramento de uma fazenda, utilizando diferentes DAGs para analisar dados, identificar emergências e registrar ocorrências automaticamente.

REFERÊNCIAS

LINK AIRFLOW -

https://airflow.apache.org/docs/apache-airflow/stable/?utm_source=chatgpt.com

LINK VIDEO AULA -

https://docs.google.com/document/d/1rAvJ0FiWZG5fgecj4VhHqDJcgQWh_O5CrmZCw812GGU/edit?tab=t.0

LINK dos EXECUTORS - <https://www.youtube.com/watch?v=TQIInLmKM4k>

TODOS OS INSIGHTs Visuais foram criados nesse Link -

https://lucid.app/lucidspark/18066335-de30-4eed-9880-54abe681d706/edit?viewport_loc=169659%2C-1344%2C12921%2C6023%2C3B0CRSot8otl&invitationId=inv_21f49f9f-9f89-43de-80e8-b16ebb14e097